# NEURAL LAYERED MIN-SUM DECODERS FOR ALGEBRAIC CODES

**Anonymous authors**
Paper under double-blind review

## ABSTRACT

In this article, we propose low-complexity neural network decoders based on the layered min-sum algorithm to decode binary algebraic codes. By generalizing the layered min-sum algorithm to its neural network counterpart, the number of iterations required for convergence is reduced. Consequently, the number of network weights decreases while retaining a good error correction performance. The Bose-Chaudhuri-Hocquenghem (BCH) codes and quadratic residue (QR) codes are selected as two exemplary binary algebraic codes. Simulation results show that the proposed decoders achieve superior performance with less computational complexity compared with the decoders proposed by Chen & Ye (2021). Further, a neural decoder incorporating the modified random redundant decoding (mRRD) algorithm is investigated to approach the performance of maximum-likelihood (ML) decoding for some short codes.

## 1 INTRODUCTION

Algebraic codes such as Bose-Chaudhuri-Hocquenghem (BCH) codes and quadratic residue (QR) codes are important in physical-layer digital communications. For example, BCH codes have been applied in satellite communications (Cheung & Pollara, 1988) and Compact Disc players. The (24, 12) extended Golay code has been used in the imaging systems for space exploration (Wicker, 1994) and high-frequency radio systems (Honary et al., 1994). In contrast to low-density parity-check (LDPC) codes, many algebraic codes are proved to have excellent performance, but they need efficient decoding algorithms to approach the theoretical performance limit. Recently, short algebraic codes have returned the spotlight again due to their good maximum-likelihood (ML) decoding performance. Accordingly, efficient algebraic decoders have been in high demand, especially for achieving near ML performance.

The idea of using machine learning to design or decode channel codes has been extensively investigated in recent years. Kim et al. (2018) showed that recurrent neural networks (RNN) can be trained to decode convolutional codes and turbo codes, and in Zhang et al. (2022); Jiang et al. (2020; 2019), data-driven end-to-end autoencoders have been utilized to learn to encode and decode simultaneously. Huang et al. (2020) used reinforcement learning and genetic algorithm to design some binary codes, which achieve equivalent performance to some existing codes and even outperform certain codes constructed by traditional methods. Among these works, an important branch of studies is the neural belief propagation (BP) based decoders. BP-based decoders can achieve near-capacity performance for decoding LDPC codes. However, the decoding performance deteriorates significantly for decoding BCH codes since the parity-check matrices of BCH codes are much denser. As an iterative message passing algorithm, the BP-based algorithms can be naturally generalized to neural networks. Nachmani et al. (2016) introduced the neural BP decoders by assigning weights to each edge of the Tanner graph and training these weights, which greatly outperforms the conventional BP algorithm for decoding BCH codes. Lugosch & Gross (2017); Nachmani et al. (2018) further reduced the complexity of neural BP by replacing the sum-product operations with min-sum functions. The authors also proposed using additive weights instead of multiplicative weights to further lower the complexity. Moreover, the modified random redundant decoding (mRRD) algorithm (Dimnik & Be'ery, 2009) was applied to enhance the performance of neural BP-based decoders. Specifically, for the BCH (63, 45, 7) code, it needs up to $50 \times 50 \times 5$ iterations to achieve the near ML performance. In addition, Nachmani & Wolf (2019) improved the error correction performance by using

hyper-graph networks to learn the weights of the edges. They also replaced the $\tanh^{-1}(\cdot)$ operations in neural BP decoders by its high order Taylor approximation to prevent gradient explosion.

It is noted that the number of performed permutations significantly contributes to the computational complexity of random redundant decoding (RRD) (Halford & Chugg, 2006) and mRRD algorithms. Instead of using randomly selected permutations, Liu et al. (2020) proposed to use clustering algorithms to classify the permutations involved in the RRD algorithm and then sort and select the permutations according to the received vectors. Moreover, they introduced a neural RRD algorithm to further improve the decoding performance. Chen & Ye (2021) proposed a cyclically equivariant neural decoder to decode cyclic codes. The authors exploited the cyclic property of the codes to reduce the number of trainable weights, and the decoding procedure is performed on cyclic redundant parity-check matrices. The cyclically equivariant neural decoder achieves better error performance than the decoders proposed by Nachmani et al. (2018) andNachmani & Wolf (2019). Additionally, the authors proposed a list decoding method (i.e., a variant of mRRD algorithm), to improve the performance of the cyclically equivariant decoders. Particularly, it needs up to $64 \times 20 \times 5$ iterations to achieve the near ML performance for the BCH (63, 45, 7) code, which is much less than that of the mRRD algorithm in Nachmani et al. (2018).

Besides, Buchberger et al. (2021) proposed to perform decoding on large redundant matrices to obtain better performance. In order to construct these huge parity-check matrices, the authors used all the codewords with the minimum weight from the dual codes of original codes as rows. Next, they trained neural networks representing these matrices and pruned the rows and the corresponding neural network at each iteration. The proposed decoder achieves near ML performance on certain Reed-Muller codes without using the list decoder or mRRD algorithm. However, the number of codewords with the minimum weight may be numerous in the dual code, and it is even difficult to calculate the minimum distances of some codes. We also point out that the computational complexity at each iteration increases with the growing dimension of parity-check matrices for the BP-based algorithms.

In order to address the concerns regarding inefficient BP-based neural decoders for binary algebraic codes, this paper proposes to construct a neural network based on the layered min-sum algorithm. The purpose is to reduce the number of operations at each iteration and to improve the neural decoding performance. Further, the mRRD algorithm is incorporated to approach the ML performance. We point out that both the proposed neural layered min-sum and neural mRRD algorithms can be conducted in parallel. To summarize, the main contributions of this paper include the following aspects.

- We propose a low complexity neural decoder based on the layered min-sum algorithm. The proposed decoding algorithm has a faster convergence speed, and it could achieve better performance with the same number of iterations and fewer trainable weights as in existing neural decoders.
- We apply the mRRD algorithm to the proposed decoder to achieve the ML performance for some codes, and we also introduce an early stopping criterion to accelerate the decoding process. The maximum and the average number of iterations for the proposed decoder on different signal-to-noise ratios (SNRs) are significantly less than that of the list decoder in Chen & Ye (2021), and the operations at each iteration of our decoder are simpler.
- We propose a simulation-based algorithm to acquire the ML performance of linear block codes of short-to-moderate lengths by utilizing linear programming and integer programming.

## 2 BACKGROUNDS

An $(n, k, d)$ linear block code is a code of length $n$, dimension $k$ and minimum hamming distance $d$. All codewords can be generated using a $k \times n$ generator matrix $\boldsymbol{G}$. Given an arbitrary binary sequence $\boldsymbol{m}$ of length $k$, the encoded codeword can be obtained as $\boldsymbol{u} = \boldsymbol{m}\boldsymbol{G}$. The decoding results can be examined by an $(n-k) \times n$ parity-check matrix $\boldsymbol{H}$. The receiver can believe that a decoding result $\hat{\boldsymbol{u}}$ is error-free if $\hat{\boldsymbol{u}}\boldsymbol{H}^T = \boldsymbol{0}$. Each parity-check matrix corresponds to a bipartite Tanner graph $(\mathbb{C} \cup \mathbb{V}, \mathbb{E})$, where the variable node set $\mathbb{V}$ corresponds to the bits in a codeword and the check node set $\mathbb{C}$ associates with the parity-check equations in $\boldsymbol{H}$. An edge between a check node and

a variable node exists if the variable node is involved in the equation corresponding to the check node, i.e., edge $e = (c_i, v_j)$ exists if $\boldsymbol{H}_{i,j} = 1$. For example, a parity-check matrix of the (7,4,3) Hamming code is

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix},$$

and the corresponding Tanner graph is shown in Figure 1.

Assume the communication channel is a binary input additive white Gaussian noise (BI-AWGN) channel. A codeword $\boldsymbol{u} = (u_1, u_2, ..., u_n) \in \{0, 1\}^n$ is sent through the channel, and then we obtain the binary phase shift keying (BPSK) modulated vector $\boldsymbol{y} = ((-1)^{u_1}, (-1)^{u_2}, ..(-1)^{u_n})$. Let the noise vector be $\mathbf{n} \in \mathbb{R}^n$, where $\mathrm{n}_i \sim \mathcal{N}(0,\, \sigma^2)$, then the received vector is $\boldsymbol{r} = \boldsymbol{y} + \mathbf{n}$. In the flooding scheduled min-sum or sum-product algorithm, the log-likelihood ratios (LLRs) are passed simultaneously and repeatedly between check nodes and variable nodes, and the decoding procedure can be described as follows.

1. Initialization:

$$l^1_{c_j \to v_i} = 0, \tag{1}$$

$$l^{ch}_i = \frac{2r_i}{\sigma^2} \text{ (for sum-product algorithm)}, \quad or \quad l^{ch}_i = r_i \text{ (for min-sum algorithm)}. \tag{2}$$

2. For $k = 1$ to $i_{max}$ (i.e., the maximum number of iterations):
   (a) Variable nodes to check nodes update:

$$l^k_{v_i \to c_j} = l^{ch}_i + \sum_{c_{j'} \in N(v_i) \setminus \{c_j\}} l^k_{c_{j'} \to v_i}, \tag{3}$$

   where $N(v_i)$ refers to the nodes that are adjacent to node $v_j$ and "\" is subtraction between two sets.
   (b) Check nodes to variable nodes update:

$$l^{k+1}_{c_j \to v_i} = \min_{v_{i'} \in N(c_j) \setminus \{v_i\}} |l^k_{v_{i'} \to c_j}| \prod_{v'_i \in N(c_j) \setminus \{v_i\}} \text{sign}(l^k_{v_{i'} \to c_j}) \text{ (for min-sum algorithm)}, \tag{4}$$

$$l^{k+1}_{c_j \to v_i} = 2\tanh^{-1}\left( \prod_{i' \in N(c_j) \setminus \{v_i\}} \tanh(l^k_{v_{i'} \to c_j}) \right) \text{ (for sum-product algorithm)}. \tag{5}$$

   (c) Output:

$$o^k_i = l^{ch}_i + \sum_{c_{j'} \in N(v_i)} l^{k+1}_{c_{j'} \to v_i}. \tag{6}$$

   (d) Stop criterion: Let the decoding results be $\hat{\boldsymbol{u}}^k = (\hat{u}^k_1, \hat{u}^k_2, \ldots, \hat{u}^k_n)$, where $\hat{u}^k_i = 0$ if $o^k_i \geq 0$ and $\hat{u_i}^k = 1$ if $o^k_i < 0$. Return the decoding result $\hat{\boldsymbol{u}}^k$ if $\hat{\boldsymbol{u}}^k \boldsymbol{H}^T = 0$ or $i_{max}$ is reached.

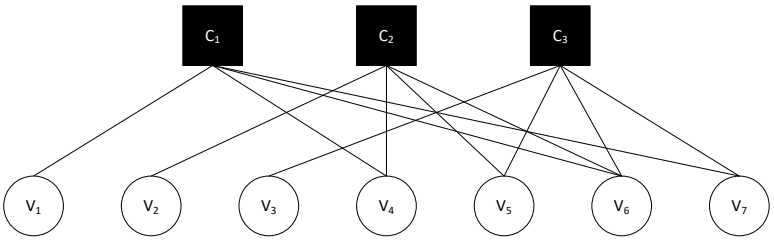



Figure 1: The Tanner graph of the (7,4,3) Hamming code.

In addition, Nachmani et al. (2018) proposed the neural normalized min-sum (NNMS) algorithm, where Equation (4) is modified as

$$l^{k+1}_{c_j \to v_i} = w^k_{v_{i'} \to c_j} \min_{v_{i'} \in N(c_j) \setminus \{v_i\}} |l^k_{v_{i'} \to c_j}| \prod_{v'_i \in N(c_j) \setminus \{v_i\}} \text{sign}(l^k_{v_{i'} \to c_j}), \tag{7}$$

and $w_{v_i \to c_j} \in (0, 1)$ is the weight of the edge $(v_i, c_j)$. Assume that there are $n_e$ edges in a Tanner graph and the neural network represents $i_{max}$ min-sum iterations, then there are $i_{max} \times n_e$ weights to be trained in the neural network. An exemplary NNMS structure is depicted in Figure 2.

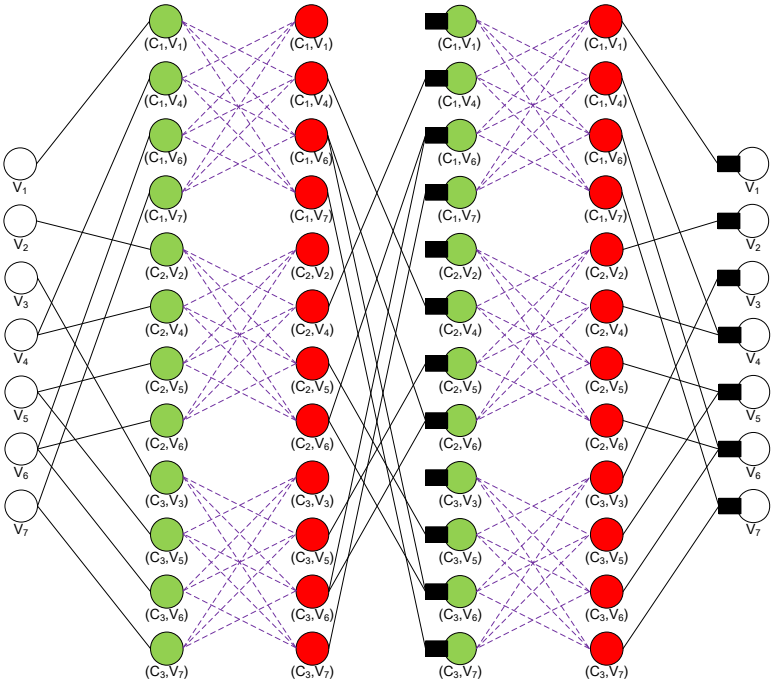



Figure 2: The exemplary NNMS structure (with 2 iterations) constructed according to Figure 1. The purple dashed lines represent trainable weights and black rectangles denote additions of the received channel LLRs.

In contrast to the flooding min-sum algorithm, the layered (or asynchronous) min-sum algorithm converges much faster while still keeping the same error performance. The reason is that the check nodes are not updated simultaneously in the layered min-sum algorithm. Therefore, the check nodes updated later could utilize the latest LLR information from the previously updated nodes at the same iteration. That motivates us to generalize the layered min-sum algorithm to its neural network counterpart, and the resultant neural decoder may achieve similar error performance with fewer iterations and trainable weights.

## 3 Neural Layered Min-sum Algorithm

The layered min-sum algorithm divides the Tanner graph into several clusters and serially performs decoding iterations within each cluster. We set iteration-wise weights for the neural layered min-sum decoder to significantly reduce the number of weights. For example, given an $(n - k) \times n$ parity-check matrix of the (63, 45, 7) BCH code, it has $n_e = 432$ edges and the row weight $n_r = 24$. If we use edge-wise weights as in the NNMS algorithm (Equation (7)), the total number of weights would be $n_e \times i_{max} = 432 \times i_{max}$. Moreover, if we use an $n \times n$ cyclic redundant matrix (Appendix B), the number would amount to $n \times n_r \times i_{max} = 1512 \times i_{max}$. As for the cyclically equivariant decoder Chen & Ye (2021), the number of weights is $n_r \times n_r \times i_{max} = 576 \times i_{max}$. On the other hand, if we employ the layer-wise weights, the total number of weights would be only $i_{max}$ regardless of the utilization of decoding matrices. The layered min-sum algorithm is described as follows, where the initialization and stopping criterion steps are the same as the min-sum algorithm in Section 2.

1. For $k = 1$ to $i_{max}$:
   (a) For $j = 1$ to $|\mathbb{C}|$:

i. Variable nodes to check nodes update:

$$l^{k+1}_{v_i \to c_j} = l^{k}_{v_i \to c_j} + \sum_{c_{j'} \in N(v_i) \setminus \{c_j\}} l^{k}_{c_{j'} \to v_i}, \text{ where } v_i \in N(c_j). \tag{8}$$

ii. Check nodes to variable nodes update:

$$l^{k}_{c_j \to v_i} = w^k \min_{v_{i'} \in N(c_j) \setminus \{v_i\}} |l^{k}_{v_{i'} \to c_j}| \prod_{v'_i \in N(c_j) \setminus \{v_i\}} \text{sign}(l^{k}_{v_{i'} \to c_j}). \tag{9}$$

(b) Output:

$$o^k_i = l^{k}_{v_i \to c_j} + \sum_{c_{j'} \in N(v_i)} l^{k+1}_{c_{j'} \to v_i}. \tag{10}$$

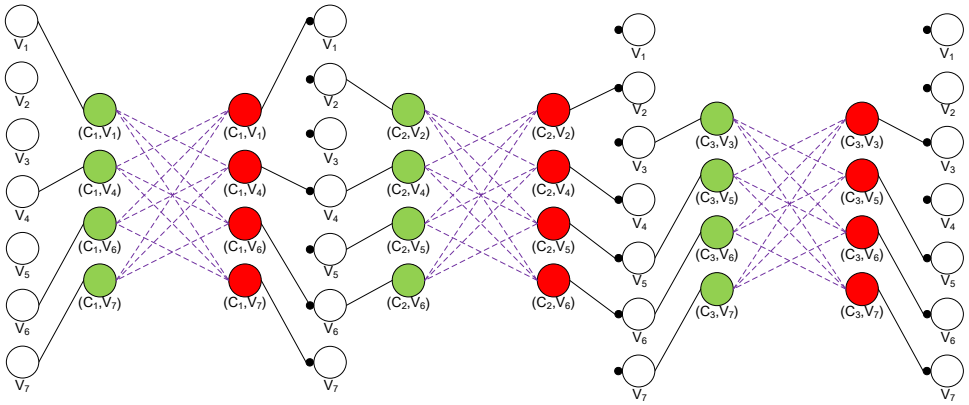



Figure 3: The exemplary structure of the proposed neural layered min-sum decoder in accordance to Figure1. The purple dashed lines correspond to trainable weights, and it is noted that they share the same values at each iteration. The black circles represent the additions of LLRs from the previous update.

The structure of the neural layered min-sum algorithm is demonstrated in Figure 3. The output vector can be interpreted as an estimation of the LLR vector, i.e. $o_i \approx \log \frac{P(u_i=0|\boldsymbol{r})}{P(u_i=1|\boldsymbol{r})}$. Therefore, the output can be converted to the probability estimation from the sigmoid function. On other words, $\sigma(o_i) \approx P(u_i = 0|\boldsymbol{r})$ and $\sigma(-o_i) \approx P(u_i = 1|\boldsymbol{r})$. In order to measure the difference between the output vector $\boldsymbol{o}$ and the labels (i.e., the true codeword $\boldsymbol{u}$), a binary cross entropy loss function is used, i.e.,

$$L(\boldsymbol{o}, \boldsymbol{u}) = -\frac{1}{n} \sum_{i=1}^{n} u_i \log(\sigma(-o_i)) + (1 - u_i) \log(\sigma(o_i)). \tag{11}$$

## 4 Modified Random Redundant Decoding Algorithm

To further enhance the error performance of the proposed neural layered min-sum algorithm, the modified redundant decoding (mRRD) algorithm is incorporated, which permutes the received vector multiple times and then decodes these permuted vectors. A simple example of permutation is described in the following. Given a permutation $\pi = \left(\begin{smallmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \end{smallmatrix}\right)$, it permutes any codeword $\mathbf{u} = (u_1, u_2, u_3)$ to $\pi(\mathbf{u}) = (u_3, u_2, u_1)$. The permutations used in the decoding process are chosen from the permutation group $\text{Per}(\mathcal{C})$, consisting of all permutations that preserve the codewords from the code ensemble $\mathcal{C}$. For instance, all cyclic shifts are in the permutation group of a cyclic code. The permutation group is defined by

$$\text{Per}(\mathcal{C}) = \{\pi | \pi(\mathbf{u}) \in \mathcal{C}, \forall \mathbf{u} \in \mathcal{C}\}. \tag{12}$$

We also apply an early stopping criterion (Kaneko et al., 1994; Wang et al., 2013) to the mRRD algorithm, and it enables the decoding procedure to quit immediately once the ML codeword is

found. The details of this stopping criterion can be found in Appendix A, and the mRRD algorithm of size $(\mathcal{W}, \mathcal{L}, i_{max})$ is summarized in Algorithm 1.

The core idea of the mRRD algorithm is to permute the input vector, and thus the input diversity grows greatly. As a consequence, the application of permutation may yield different outputs and increases the probability to produce a correct result. Moreover, by permuting the codewords during the decoding procedure, it can effectively prevent the propagation of false information caused by trapping sets (Wang et al., 2022). Finally, we can choose the best estimation from all the different decoding results by calculating their correlation discrepancies $\lambda(\boldsymbol{r}, \hat{\boldsymbol{u}})$ using the received vector $\boldsymbol{r}$. The definition of correlation discrepancy is given in Appendix A.

**Algorithm 1** mRRD$(\mathcal{W}, \mathcal{L}, i_{max})$

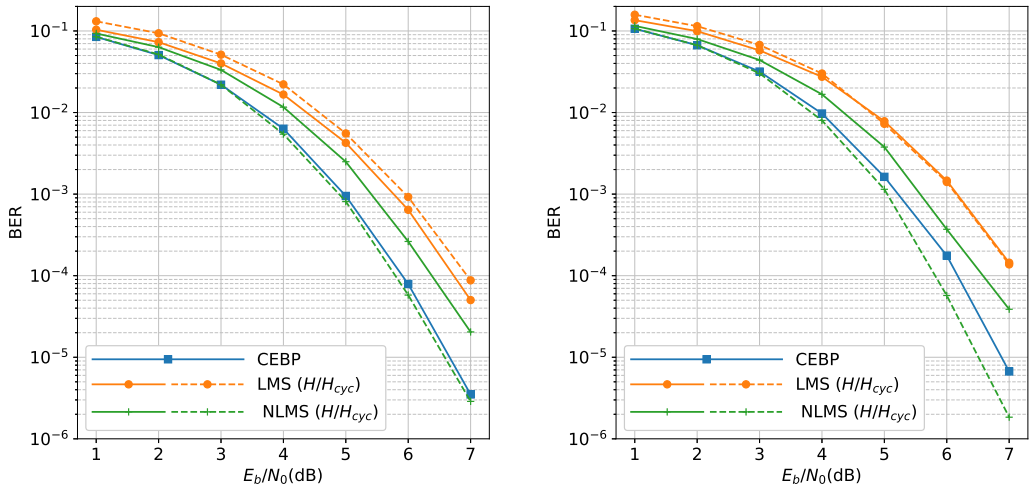
**Input:** Received vector $\boldsymbol{r}$
**Output:** Decoded vector $\hat{\boldsymbol{u}}$
$\boldsymbol{o} = \boldsymbol{r}, \mathbb{S} = \emptyset$
**for** $i \in \{1, \ldots, \mathcal{W}\}$ **do**
  **for** $j \in \{1, \ldots, \mathcal{L}\}$ **do**
    Randomly draw a permutation $\pi$ from Per$(\mathcal{C})$
    $\boldsymbol{o} =$ Min-sum$(\pi(\boldsymbol{o}))$ // Perform min-sum decoding with input $\pi(\boldsymbol{o})$, up to $i_{max}$ iterations
    $\hat{\boldsymbol{u}} =$ Hard-Decision$(\boldsymbol{o})$
    $\boldsymbol{o} = \pi^{-1}(\boldsymbol{o}), \hat{\boldsymbol{u}} = \pi^{-1}(\hat{\boldsymbol{u}})$ //Reorder the bits to its original (unpermuted) order.
  **end for**
  **if** $\hat{\boldsymbol{u}}\boldsymbol{H}^T = 0$ **then**
    $\mathbb{S} = \mathbb{S} \cup \{\hat{\boldsymbol{u}}\}$ //Append a successfully decoded result to the candidate codeword set
  **end if**
  **if** $\hat{\boldsymbol{u}}$ is the ML codeword **then**
    return $\hat{\boldsymbol{u}}$ //End the decoding process immediately if the ML codeword is found
  **end if**
**end for**
**if** $|\mathbb{S}| \neq 0$ **then**
  return $\text{argmin}_{\hat{\boldsymbol{u}} \in \mathbb{S}} \lambda(\boldsymbol{r}, \hat{\boldsymbol{u}})$ //Return the most likely codeword if there is no ML codeword identified.
**else**
  return $\mathbf{0}$ //Return an all-zero codeword if there is no codeword found.
**end if**

## 5 Simulation Results

In the following sections, we will use the acronym "CEBP" to refer to the cyclically equivariant BP decoders in Chen & Ye (2021), and the acronyms "LMS" and "NLMS" denote the layered min-sum and neural layered min-sum algorithms, respectively. It is worth mentioning that the CEBP can only be applied to cyclic redundant matrices. We use $\boldsymbol{H}$ and $\boldsymbol{H}_{cyc}$ to represent the original matrix and the cyclic redundant matrix, respectively. More information on $\boldsymbol{H}_{cyc}$ is detailed in Appendix B. Similarly to other neural BP-based decoders, e.g., Nachmani et al. (2018) and Buchberger et al. (2021), our proposed decoder is trained using all-zero codewords corrupted by random noise and $i_{max} = 5$ for all decoders. After training, the randomly generated codewords and noise are used for performance evaluation. All reported data points are obtained by simulation until no less than 100 error frames occur.

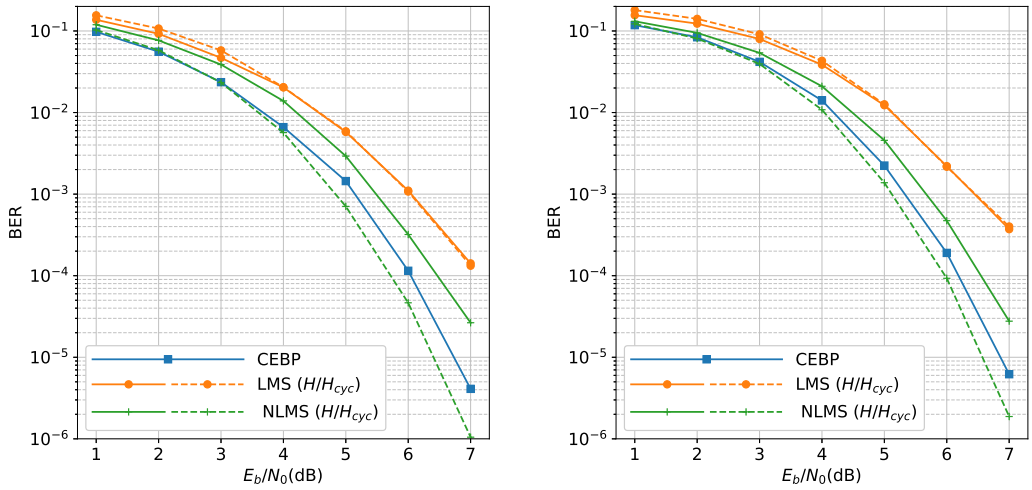
### 5.1 Performance of NLMS Decoding

Simulation results of the (63,45,7) BCH code are shown in Figure 4a. It can be observed that the bit error rate (BER) difference of the LMS between two matrices (i.e., $\boldsymbol{H}$ and $\boldsymbol{H}_{cyc}$) is small. However, the NLMS decoder achieves a significant gain over the LMS counterpart in both cases of $\boldsymbol{H}$ and $\boldsymbol{H}_{cyc}$, and the improvement is 0.35 dB and 1.12 dB, respectively at BER=$10^{-4}$. Moreover, it is found that the NLMS decoder with $\boldsymbol{H}_{cyc}$ significantly outperforms the one using the original matrix $\boldsymbol{H}$, and this phenomenon is also observed for other codes. Additionally, the performance gain of the




(a) BCH (63,45,7) (b) BCH (63,36,11)

Figure 4: The error performance of different decoders on the BCH(63,45,7) and BCH(63,36,11) codes.




(a) QR (47,24,11) (b) QR (71,36,11)

Figure 5: The error performance of different decoders on the QR(47,24,11) and QR(71,36,11) codes.

NLMS decoder over the CEBP decoder is 0.06 dB at BER=$4 \times 10^{-6}$. The performance comparisons of different decoders on the (63,36,11) BCH code are illustrated in Figure 4b. It is noted that the NLMS decoder achieves a gain of 0.58 dB over the LMS decoder in the case of $\boldsymbol{H}$, respectively 1.20 dB in the case of $\boldsymbol{H}_{cyc}$ at BER=$2 \times 10^{-4}$. Further, we found that the NLMS decoder outperforms the CEBP counterpart by 0.37 dB at BER=$10^{-5}$.

The BER performance of different decoders on the (47,24,11) QR code is plotted in Figure 5a. It can be seen that the NLMS decoder achieves a gain of 0.64 dB over the LMS decoder using $\boldsymbol{H}$, and a gain of 1.18 dB in the case of $\boldsymbol{H}_{cyc}$ for BER=$2 \times 10^{-4}$. We also observe that the NLMS outperforms the CEBP decoder by 0.32 dB at BER=$10^{-5}$. The BER curves of different decoders on the (71,36,11) QR code are depicted in Figure 5b. The NLMS decoder achieves a gain of 0.90 dB over the LMS decoder with $\boldsymbol{H}$ and a gain of 1.38 dB in the case of $\boldsymbol{H}_{cyc}$ at BER=$4 \times 10^{-4}$. Moreover, it is noticed that the NLMS decoder outperforms the CEBP decoder by 0.29 dB at BER=$10^{-5}$.

Table 1: The number of operations for different decoders on the BCH(63,45,7) code.

| Algorithm | + | $\times$ | $\tanh(\cdot)$ | $\tanh^{-1}(\cdot)$ | sign flips | min |
|---|---|---|---|---|---|---|
| CEBP | 1,449 | 4,410 | 63 | 63 | 0 | 0 |
| NLMS | 1,449 | 1512 | 0 | 0 | 1386 | 63 |

### 5.2 Performance of Neural mRRD Algorithm

The ML decoding performance of BCH codes in this section is cited from the database in Helmling et al. (2019). The ML performance of QR codes is obtained by simulation, and the details are described in Appendix C. The "List decoder" refers to the list decoders with CEBP decoder proposed by Chen & Ye (2021) and the term "Neural mRRD" refers to our proposed neural mRRD Algorithm 1. The size of a decoder is denoted by $(\mathcal{W}, \mathcal{L}, i_{max})$, implying the required number of iterations in the worst case is $\mathcal{W} \times \mathcal{L} \times i_{max}$.

It can be seen from Figure 6 that with the growing width $\mathcal{W}$ of decoders, both the mRRD and list decoders can achieve near ML performance. For the mRRD and list decoder with the best performance, the worst-case complexity of the list decoder is 5 times as much as the mRRD algorithm (i.e., $64 \times 20 \times 5 = 5 \times (256 \times 5 \times 1)$). In addition, our mRRD(256,5,1) algorithm achieves a gain of 0.34 dB over the list decoder(16,20,5) with the comparable complexity. Moreover, the mRRD(16,5,1) and mRRD(32,5,1) algorithms achieve a similar performance of list decoder(8,20,5) and list decoder(16,20,5), respectively, but the number of iterations is reduced significantly by 90%.

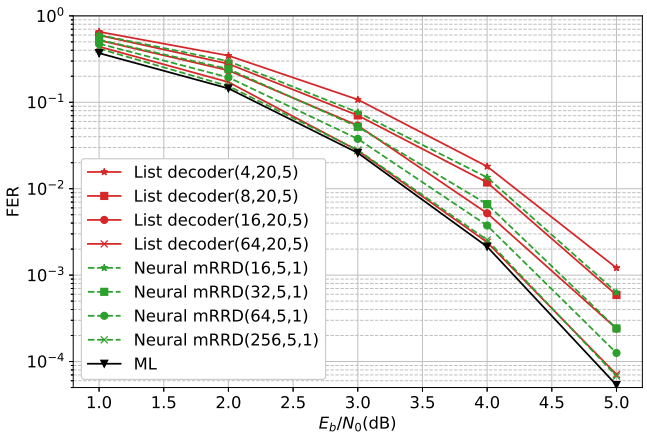



Figure 6: Frame error rate (FER) performance of the BCH (63,45,7) code.

The FER performance of the mRRD algorithm with different sizes for the (47,24,11) QR code is shown in Figure 7. The performance of the list decoder on the QR code is absent since the affine permutations used in the list decoder cannot be applied directly to the QR code. Notably, one can see that the mRRD(128,5,1) algorithm approaches the optimal ML decoding performance.

## 6 Complexity Analysis

Given an $n \times n$ cyclic redundant matrix $\boldsymbol{H}_{cyc}$, let $n_r$ be its row weight, then the column weight of $\boldsymbol{H}_{cyc}$ is also $n_r$. For each iteration, the CEBP decoder needs $n(n_r - 1)$ additions, $n(3n_r - 2)$ multiplications, $n$ times of $\tanh(\cdot)$ operations and $n$ times of $\tanh^{-1}(\cdot)$ operations. In comparison, each NLMS decoding iteration consists of $n(n_r - 1)$ additions, $n \times n_r$ multiplications, $n(n_r - 2)$ times of sign flips and $n$ times of $\min(\cdot)$ operations. Table 1 illustrates the specific numbers of each operation per iteration on the (63,45,7) BCH code.

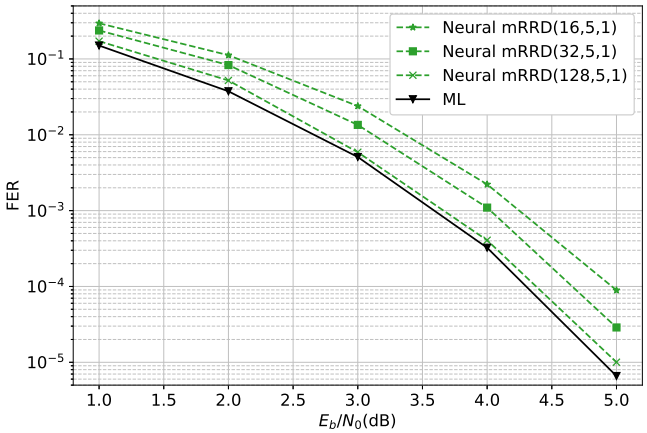



Figure 7: Decoding performance of the QR (47,24,11) code.

The complexity of the list decoder and the proposed mRRD algorithm is evaluated by simulating 10,000 frames for each SNR and then calculating the average number of iterations as shown in Figure 8. Recall that the maximum number of iterations of the mRRD algorithm is only 1/5 of that of the list decoder when the similar near ML performance is attained. Now it can be noticed from Figure 8 that the complexity reduction is further magnified with increasing SNR, which attributes to the early stopping criterion in the proposed mRRD algorithm.

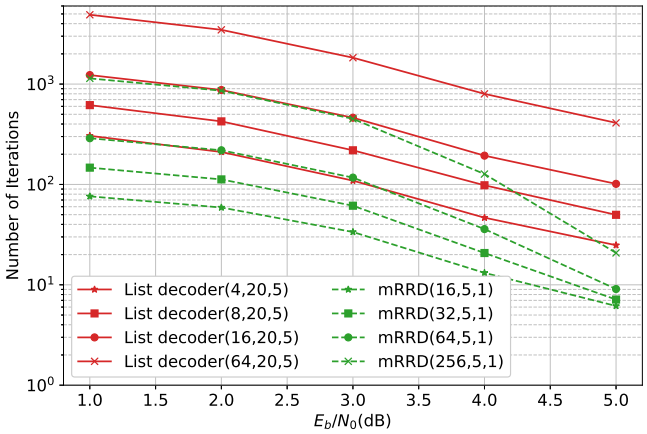



Figure 8: Comparison of decoding complexity on the BCH (63,45,7) code.

## 7 Conclusion

This paper proposes a neural layered min-sum (NLMS) decoding algorithm to decode binary algebraic codes, and we take BCH codes and QR codes as examples. Using the cyclic redundant matrix, we observed that the performance of the NLMS decoding could be greatly improved, whereas the performance improvement for the conventional layered min-sum algorithm is limited. Moreover, the proposed NLMS decoder with the cyclic redundant matrix consistently outperforms the neural decoders proposed by Chen & Ye (2021), and it also has lower computational complexity. In order to approach the maximum likelihood (ML) decoding performance, we further combine the mRRD method with the proposed NLMS decoder. The resultant neural mRRD algorithm achieves near ML performance for certain codes and reaches a significant reduction of computations compared to the list decoder in Chen & Ye (2021).

## REFERENCES


Andreas Buchberger, Christian Häger, Henry D. Pfister, Laurent Schmalen, and Alexandre Graell i Amat. Pruning and quantizing neural belief propagation decoders. *IEEE Journal on Selected Areas in Communications*, 39(7):1957–1966, 2021. doi: 10.1109/JSAC.2020.3041392.

Xiangyu Chen and Min Ye. Cyclically equivariant neural decoders for cyclic codes. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pp. 1771–1780. PMLR, 2021.

K-M Cheung and F Pollara. Phobos lander coding system: Software and analysis. *The Telecommunications and Data Acquisition Report*, 1988.

Ilan Dimnik and Yair Be'ery. Improved random redundant iterative HDPC decoding. *IEEE Transactions on Communications*, 57(7):1982–1985, July 2009. doi: 10.1109/TCOMM.2009.07.070621.

Jon Feldman, Martin J. Wainwright, and David R. Karger. Using linear programming to decode binary linear codes. *IEEE Transactions on Information Theory*, 51(3):954–972, March 2005. doi: 10.1109/TIT.2004.842696.

Thomas R. Halford and Keith M. Chugg. Random redundant soft-in soft-out decoding of linear block codes. In *2006 IEEE International Symposium on Information Theory*, pp. 2230–2234, 2006. doi: 10.1109/ISIT.2006.261947.

Michael Helmling, Stefan Scholl, Florian Gensheimer, Tobias Dietz, Kira Kraft, Stefan Ruzika, and Norbert Wehn. Database of channel codes and ML simulation results., 2019. URL `www.uni-kl.de/channel-codes`.

Bahram Honary, Bernard Hunt, and M. Maundrell. Improving automatic link establishment through a new soft decision trellis decoder for the (24,12) Golay code. In *1994 Sixth International Conference on HF Radio Systems and Techniques*, pp. 182–185, 1994. doi: 10.1049/cp:19940489.

Lingchen Huang, Huazi Zhang, Rong Li, Yiqun Ge, and Jun Wang. Ai coding: Learning to construct error correction codes. *IEEE Transactions on Communications*, 68(1):26–39, 2020. doi: 10.1109/TCOMM.2019.2951403.

Yihan Jiang, Hyeji Kim, Himanshu Asnani, Sreeram Kannan, Sewoong Oh, and Pramod Viswanath. Turbo autoencoder: Deep learning based channel codes for point-to-point communication channels. In *NeurIPS*, pp. 2754–2764, 2019.

Yihan Jiang, Hyeji Kim, Himanshu Asnani, Sreeram Kannan, Sewoong Oh, and Pramod Viswanath. Learn codes: Inventing low-latency codes via recurrent neural networks. *IEEE Journal on Selected Areas in Information Theory*, 1(1):207–216, 2020. doi: 10.1109/JSAIT.2020.2988577.

Toshimitsu Kaneko, Toshihisa Nishijima, Hiroshige Inazumi, and Shigeichi Hirasawa. An efficient maximum-likelihood-decoding algorithm for linear block codes with algebraic decoder. *IEEE Transactions on Information Theory*, 40(2):320–327, 1994. doi: 10.1109/18.312155.

Hyeji Kim, Yihan Jiang, Ranvir B. Rana, Sreeram Kannan, Sewoong Oh, and Pramod Viswanath. Communication algorithms via deep learning. In *International Conference on Learning Representations*, 2018.

Bryan Liu, Yixuan Xie, and Jinhong Yuan. A deep learning assisted node-classified redundant decoding algorithm for bch codes. *IEEE Transactions on Communications*, 68(9):5338–5349, 2020. doi: 10.1109/TCOMM.2020.3001162.

Loren Lugosch and Warren J. Gross. Neural offset min-sum decoding. In *2017 IEEE International Symposium on Information Theory (ISIT)*, pp. 1361–1365, 2017. doi: 10.1109/ISIT.2017.8006751.

Eliya Nachmani and Lior Wolf. Hyper-graph-network decoders for block codes. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

Eliya Nachmani, Yair Be'ery, and David Burshtein. Learning to decode linear codes using deep learning. In *2016 54th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pp. 341–346, 2016. doi: 10.1109/ALLERTON.2016.7852251.

Eliya Nachmani, Elad Marciano, Loren Lugosch, Warren J. Gross, David Burshtein, and Yair Be'ery. Deep learning methods for improved decoding of linear codes. *IEEE Journal of Selected Topics in Signal Processing*, 12(1):119–131, Feb. 2018. doi: 10.1109/JSTSP.2017.2788405.

Lin Wang, Yong Li, Trieu-Kien Truong, and Tsung-Ching Lin. On decoding of the (89, 45, 17) quadratic residue code. *IEEE Transactions on Communications*, 61(3):832–841, 2013. doi: 10.1109/TCOMM.2012.122712.120287.

Ming Wang, Yong Li, Rui Liu, Huihui Wu, Youqiang Hu, and Francis C. M. Lau. Decoding quadratic residue codes using deep neural networks. *Electronics*, 11(17), 2022. ISSN 2079-9292. doi: 10.3390/electronics11172717.

Stephen B. Wicker. *Error Control Systems for Digital Communication and Storage*. Prentice-Hall, Inc., USA, 1994. ISBN 0132008092.

Xiaojie Zhang and Paul H. Siegel. Adaptive cut generation algorithm for improved linear programming decoding of binary linear codes. *IEEE Transactions on Information Theory*, 58(10): 6581–6594, Oct. 2012. doi: 10.1109/TIT.2012.2204955.

Yitian Zhang, Huihui Wu, and Mark Coates. On the design of channel coding autoencoders with arbitrary rates for ISI channels. *IEEE Wireless Communications Letters*, 11(2):426–430, 2022. doi: 10.1109/LWC.2021.3131848.

## A EARLY STOPPING CRITERION

To determine whether a codeword is an ML codeword or not, we utilize the sufficient condition given in Kaneko et al. (1994) and Wang et al. (2013), as described in the sequel. Given the received vector $\boldsymbol{r} = (r_1, r_2, \ldots, r_n)$ and a candidate codeword $\hat{\boldsymbol{u}} = (\hat{u_1}, \hat{u_2}, \ldots, \hat{u_n})$, let

$$z_i = \begin{cases} 0 \text{ , if } r_i \geq 0 \\ 1 \text{ , if } r_i < 0 \end{cases} , \tag{13}$$

$$\mathbb{D}_0(\hat{\boldsymbol{u}}) = \{i \,|\hat{u}_i = z_i\}, \;\; \mathbb{D}_1(\hat{\boldsymbol{u}}) = \{i \,|\hat{u}_i \neq z_i\} \,. \tag{14}$$

The correlation discrepancy is defined as

$$\lambda(\boldsymbol{r}, \hat{\boldsymbol{u}}) = \sum_{i \in \mathbb{D}_1(\hat{\boldsymbol{u}})} |r_i| \,. \tag{15}$$

Assume there are $n_{\hat{\boldsymbol{u}}}$ elements in $\mathbb{D}_1(\hat{\boldsymbol{u}})$, then the index set $\mathbb{D}_0(\hat{\boldsymbol{u}})$ has $n - n_{\hat{\boldsymbol{u}}}$ elements, and these elements are sorted in ascending order of reliability. In other words, $\mathbb{D}_0(\hat{\boldsymbol{u}}) = \{l_1, l_2, \ldots, l_{n-n_{\hat{\boldsymbol{u}}}}\}$ and $|r_{l_i}| \leq |r_{l_j}|$ for any $1 \leq i < j \leq n - n_{\hat{\boldsymbol{u}}}$. Let the first $j$ elements in $\mathbb{D}_0(\hat{\boldsymbol{u}})$ be denoted by

$$\mathbb{D}_0^{(j)}(\hat{\boldsymbol{u}}) = \{l_1, l_2, \ldots, l_j\} \,. \tag{16}$$

Define $\delta = d - n_{\hat{\boldsymbol{u}}}$, where $d$ is the minimum Hamming distance of the code and $G_T(\hat{\boldsymbol{u}}, d) = \sum_{i \in \mathbb{D}_0^{(\delta)}(\hat{\boldsymbol{u}})} |r_i|$. If the condition

$$\lambda(\boldsymbol{r}, \hat{\boldsymbol{u}}) \leq G_T(\hat{\boldsymbol{u}}, d) \tag{17}$$

holds, then the decoded vector $\hat{\boldsymbol{u}}$ is the ML codeword of $\boldsymbol{r}$.

## B CYCLIC REDUNDANT PARITY-CHECK MATRIX

An original parity-check matrix of an $(n, k, d)$ cyclic code is an $(n-k) \times n$ matrix, as shown in Equation (18), where ">>" refers to the right circular shift and $\boldsymbol{H}_1$ is the first row of $\boldsymbol{H}$.

$$\boldsymbol{H} = \begin{bmatrix} \boldsymbol{H}_1 \\ \boldsymbol{H}_1 >> 1 \\ \boldsymbol{H}_1 >> 2 \\ \cdots \\ \boldsymbol{H}_1 >> (n-k-1) \end{bmatrix}. \tag{18}$$

On the contrary, the cyclic redundant matrix is an $n \times n$ matrix. Let $\boldsymbol{H}_x$ be the $x$-th row of $\boldsymbol{H}$, with the first element of $\boldsymbol{H}_x$ being 1. The cyclic redundant matrix can then be expressed as

$$\boldsymbol{H}_{cyc} = \begin{bmatrix} \boldsymbol{H}_x \\ \boldsymbol{H}_x >> 1 \\ \boldsymbol{H}_x >> 2 \\ \cdots \\ \boldsymbol{H}_x >> (n-1) \end{bmatrix}. \tag{19}$$

## C LINEAR PROGRAMMING DECODING

It is known that the ML decoding for any binary linear block code can be formulated as an optimization problem, i.e.,

$$\min \boldsymbol{l}\boldsymbol{u}^T, \qquad \text{s.t. } \boldsymbol{u} \in \mathcal{C}\,, \tag{20}$$

where $\boldsymbol{l}$ is the log-likelihood ratio (LLR) vector and $\mathcal{C}$ is the codebook. It is clear that the original minimization (Equation (20)) is an integer programming problem. To approximate the ML decoding process, Feldman et al. (2005) relaxed the feasible space by changing the whole codebook $\mathcal{C}$ to the convex hull of $\mathcal{C}$, which is called the codeword polytope. As a result, the relaxed optimization problem can be solved by linear programming (LP). If the LP decoder obtains an integer solution, then the solution is guaranteed to be an ML codeword. We point out that Zhang & Siegel (2012) proposed an efficient LP-based decoder called the adaptive cut generation-adaptive linear programming (ACG-ALP) decoder.

In this paper, a simulation-based algorithm is developed to obtain the ML decoding performance based on the ACG-ALP algorithm. It can be summarized as follows.

1. Decode the received vector using the ACG-ALP decoder. If an integer solution is obtained, then stop and return the result.
2. Utilize all the constraints of the ACG-ALP decoder as the cut tool to construct an integer programming problem and solve it.

The above approach is based on the fact that the result given by the ACG-ALP algorithm is close to the ML codeword, even though the result itself is not a true codeword. Next, by applying the integer programming after completing the ACG-ALP decoding, it would be much faster than directly finding an ML codeword with integer programming optimization from the received vector.